МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп’ютерних технологій

ЗВІТ

З ЛАБОРАТОРНОЇ №6

з дисципліни **“Функціональне програмування”**

на тему **“ЗАМИКАННЯ ТА ДЕКОРАТОРИ ЯК HOF”**

Варіант 14. Планувальник повторів задач

Виконала:
Студентка групи ФЕП-23
Поведа Анастасія

Перевірив:
проф. Свелеба С. А.

Львів-2026

Мета: навчитися будувати замикання, що зберігають стан і параметри, та параметризовані декоратори як функції вищого порядку; реалізувати декоратори вимірювання часу, повторних спроб і кешування, а за варіантом 14 — планувальник повторів задач, у якому політика затримки (фіксований або експоненційний backoff) передається функцією, а `sleep` замінено на `fake_sleep`; забезпечити детерміновані тести без реальних затримок.

Теоретичні відомості:

Замикання — функція, що захоплює змінні з області видимості, в якій її створено, і зберігає доступ до них після виходу з неї. Якщо внутрішня функція лише читає захоплені значення (параметри backoff, префікс логера), достатньо звичайного посилання; щоб змінювати стан (лічильник викликів, історія пауз), використовують `nonlocal` або мутабельний контейнер. Саме завдяки замиканням працюють фабрики функцій: `exponential_delay(base, factor)` повертає функцію `attempt → seconds`, яка «пам'ятає» свої параметри. Декоратор — функція вищого порядку, що приймає функцію і повертає її обгорнуту версію; параметризований декоратор — це фабрика, яка спочатку отримує налаштування, а вже потім саму функцію.

Декоратор `functools.wraps` копіює `__name__`, `__doc__` і виставляє `__wrapped__`, щоб обгортка не ховала оригінал. Типізація через `ParamSpec` і `TypeVar` зберігає сигнатуру для mypy. Зовнішні залежності — годинник, `sleep`, логер, джерело часу, політика затримки — інжектуються як `Callable`, тому тести підміняють їх фейками: планувальник з експоненційним backoff перевіряється миттєво, а історія запланованих пауз порівнюється з очікуваною. Декоратори застосовуються знизу вгору, тому `timed` зовні `retries` вимірює весь план повторів, а не кожну спробу.

Хід роботи:

1. У `closures.py` оголосила тип політики затримки `DelayPolicy = Callable[[int], float]` (номер невдалої спроби → пауза) і реалізувала базові

замикання: `make_counter` тримає змінний стан через `nonlocal`, `make_logger` захоплює префікс і інжектований приймач `sink`, тому в ядрі немає `print`:

```python
DelayPolicy = Callable[[int], float]
"""Номер невдалої спроби (1, 2, ...) -> пауза перед наступною спробою, с."""


def make_counter(start: int = 0) -> Callable[[], int]:
    """Лічильник викликів: стан value живе в замиканні та змінюється через nonlocal."""
    value = start

    def inc() -> int:
        nonlocal value
        value += 1
        return value

    return inc


def make_logger(prefix: str, sink: Callable[[str], None]) -> Callable[[str], None]:
    """Логер з інжектованим приймачем sink (без print у ядрі)."""
    return lambda msg: sink(f"[{prefix}] {msg}")

```

2. За варіантом 14 реалізувала фабрики політик затримок: `fixed_delay` повертає однакову паузу після кожної невдачі, `exponential_delay` — `base * factor^(attempt-1)` з верхньою межею `max_delay`; обидві валідують параметри. `make_fake_sleep` повертає пару замикань `sleep` / `slept` над спільною історією: підмінений `sleep` нічого не чекає, а лише записує тривалість:

```python
def fixed_delay(seconds: float) -> DelayPolicy:
    """Фіксований backoff: однакова пауза після кожної невдалої спроби."""
    if seconds < 0:
        raise ValueError("delay must be >= 0")
    return lambda attempt: seconds


def exponential_delay(
    base: float, factor: float = 2.0, max_delay: float = float("inf")
) -> DelayPolicy:
    """Експоненційний backoff: base * factor**(attempt-1), але не більше max_delay."""
    if base < 0 or factor <= 0 or max_delay < 0:
        raise ValueError("base, max_delay must be >= 0 and factor > 0")
    return lambda attempt: min(base * factor ** (attempt - 1), max_delay)


def make_fake_sleep(
    logger: Callable[[str], None] = lambda _: None,
) -> tuple[Callable[[float], None], Callable[[], tuple[float, ...]]]:
    """Підміна time.sleep для тестів: пара (sleep, slept) над спільним станом.

    sleep нічого не чекає, лише записує тривалість; slept повертає знімок історії.
    """
    history: list[float] = []

    def sleep(seconds: float) -> None:
        history.append(seconds)
        logger(f"(sleep {seconds:.3f}s skipped)")

    def slept() -> tuple[float, ...]:
        return tuple(history)

    return sleep, slept
```

3. У `decorators.py` описала типи: `ParamSpec` / `TypeVar`, політику без паузи `NO_DELAY` як модульну константу, протоколи `Decorator` і `CacheDecorator` з

узагальненим `__call__` (щоб `mypy --strict` виводив сигнатуру обгорнутої функції) і протокол `CachedFn` з методами керування кешем:

```python
P = ParamSpec("P")
R = TypeVar("R")
R_co = TypeVar("R_co", covariant=True)

Key = tuple[tuple[Any, ...], tuple[tuple[str, Any], ...]]
NO_DELAY: DelayPolicy = fixed_delay(0.0)


class Decorator(Protocol):
    """Декоратор, що зберігає сигнатуру: Callable[P, R] -> Callable[P, R]."""

    def __call__(self, func: Callable[P, R], /) -> Callable[P, R]: ...


class CachedFn(Protocol[P, R_co]):
    """Обгортка з кешем: звичайний виклик плюс методи керування кешем."""

    __name__: str
    __doc__: str | None

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...

    def cache_clear(self) -> None: ...

    def cache_invalidate(
        self, predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool]
    ) -> int: ...

    def cache_size(self) -> int: ...


class CacheDecorator(Protocol):
    """Декоратор, що повертає обгортку з методами кешу."""

    def __call__(self, func: Callable[P, R], /) -> CachedFn[P, R]: ...

```

4. Реалізувала параметризований декоратор `timed`: мітка, одиниці виміру, інжектовані `logger` і `clock`; вимір у блоці `finally`, тому час логується навіть при винятку; `functools.wraps` зберігає метадані:

```python
def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None],
    clock: Callable[[], float] = time.perf_counter,
) -> Decorator:
    """Логує час виконання (навіть при винятку); годинник і логер інжектовані."""
    factor = {"s": 1.0, "ms": 1e3, "us": 1e6}[unit]

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            t0 = clock()
            try:
                return func(*args, **kwargs)
            finally:
                logger(f"[timed] {name} took {(clock() - t0) * factor:.3f} {unit}")

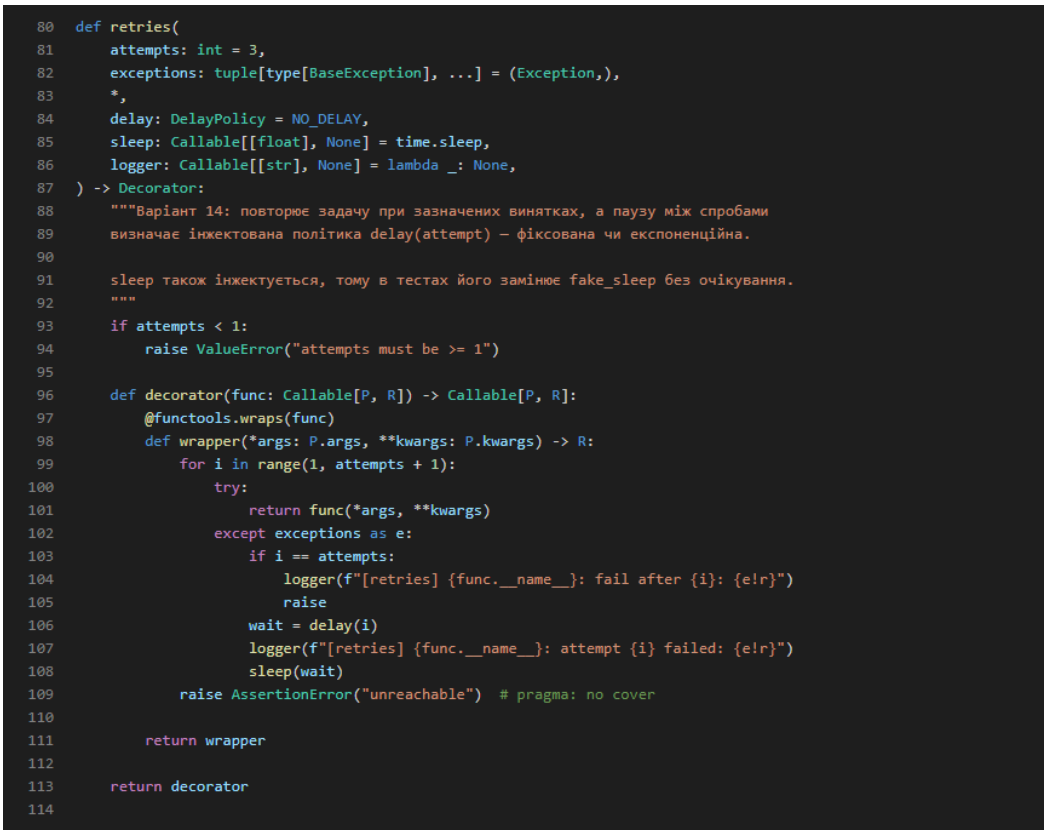
        return wrapper

    return decorator

```

5. Реалізувала `retries` — ядро планувальника повторів: кортеж винятків, на які реагуємо, інжектовані `sleep` і `logger`, а паузу перед наступною спробою

визначає інжектована політика `delay(attempt)`; після останньої невдалої спроби виняток піднімається повторно без зайвої паузи:



```python
def retries(
    attempts: int = 3,
    exceptions: tuple[type[BaseException], ...] = (Exception,),
    *,
    delay: DelayPolicy = NO_DELAY,
    sleep: Callable[[float], None] = time.sleep,
    logger: Callable[[str], None] = lambda _: None,
) -> Decorator:
    """Варіант 14: повторює задачу при зазначених винятках, а паузу між спробами
    визначає інжектована політика delay(attempt) — фіксована чи експоненційна.

    sleep також інжектується, тому в тестах його замінює fake_sleep без очікування.
    """
    if attempts < 1:
        raise ValueError("attempts must be >= 1")

    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            for i in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if i == attempts:
                        logger(f"[retries] {func.__name__}: fail after {i}: {e!r}")
                        raise
                    wait = delay(i)
                    logger(f"[retries] {func.__name__}: attempt {i} failed: {e!r}")
                    sleep(wait)
            raise AssertionError("unreachable")  # pragma: no cover

        return wrapper

    return decorator

```

6. Реалізувала `cached` з TTL: ключ з позиційних і відсортованих іменованих аргументів, протухання за інжектованим `time_fn`, блокування для безпеки у

потоках, а на обгортці — `cache_clear`, `cache_invalidate(predicate)` і `cache_size`; результат приводиться до протоколу `CachedFn` через `cast`:

```python
def make_key(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Key:
    """Ключ кешу для хешованих аргументів: (args, відсортовані kwargs)."""
    return (args, tuple(sorted(kwargs.items())))


def cached(
    *,
    ttl: float | None = None,
    time_fn: Callable[[], float] = time.time,
) -> CacheDecorator:
    """Кеш з TTL; на обгортці є cache_clear, cache_invalidate, cache_size."""

    def decorator(func: Callable[P, R]) -> CachedFn[P, R]:
        cache: dict[Key, tuple[float | None, R]] = {}
        lock = threading.Lock()  # ponytail: один лок на функцію, достатньо для ЛР

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            key = make_key(args, kwargs)
            now = time_fn()
            with lock:
                if key in cache:
                    expiry, value = cache[key]
                    if expiry is None or now < expiry:
                        return value
                    del cache[key]
            value = func(*args, **kwargs)
            with lock:
                cache[key] = (None if ttl is None else now + ttl, value)
            return value

        def cache_clear() -> None:
            with lock:
                cache.clear()

        def cache_invalidate(
            predicate: Callable[[tuple[Any, ...], dict[str, Any]], bool],
        ) -> int:
            with lock:
                doomed = [k for k in cache if predicate(k[0], dict(k[1]))]
                for k in doomed:
                    del cache[k]
            return len(doomed)

        def cache_size() -> int:
            with lock:
                return len(cache)

        wrapper.cache_clear = cache_clear  # type: ignore[attr-defined]
        wrapper.cache_invalidate = cache_invalidate  # type: ignore[attr-defined]
        wrapper.cache_size = cache_size  # type: ignore[attr-defined]
        return cast(CachedFn[P, R], wrapper)

    return decorator
```

7. В `app.py` зібрала демонстрацію планувальника: `sync_task` під `@timed` і `@retries` з фіксованим backoff 0.5 с падає двічі, `download_task` з експоненційним backoff 0.1 → 0.2 → 0.4 (межа 0.5) повторюється лише при `TimeoutError`,

`expensive_report` кешується; годинник детермінований через замикання-лічильник, а `fake_sleep` накопичує історію пауз:

```python
log = make_logger("app", print)
ticks = make_counter()
fake_clock = lambda: ticks() * 0.001  # noqa: E731  — детермінований «годинник»
fake_sleep, slept = make_fake_sleep(log)

flaky_calls = make_counter()
download_calls = make_counter()


@timed("sync", logger=log, clock=fake_clock)
@retries(attempts=4, delay=fixed_delay(0.5), sleep=fake_sleep, logger=log)
def sync_task() -> str:
    """Задача з фіксованим backoff: перші два виклики падають, третій успішний."""
    if flaky_calls() < 3:
        raise ConnectionError("sync endpoint busy")
    return "synced"


@timed("download", logger=log, clock=fake_clock)
@retries(
    attempts=5,
    exceptions=(TimeoutError,),
    delay=exponential_delay(0.1, factor=2.0, max_delay=0.5),
    sleep=fake_sleep,
    logger=log,
)
def download_task(url: str) -> str:
    """Задача з експоненційним backoff (0.1, 0.2, 0.4 → 0.5 cap)."""
    if download_calls() < 4:
        raise TimeoutError(f"timeout {url}")
    return f"content of {url}"


@cached(ttl=60.0, time_fn=fake_clock)
def expensive_report(day: int) -> str:
    """Дорога операція з кешем — повторне планування не перераховує її."""
    log(f"building report for day {day}")
    return f"report-{day}"


def main() -> None:
    """Запускає задачі планувальника і показує історію пауз без очікування."""
    print("sync:", sync_task())
    print("download:", download_task("https://example.org/data"))
    print("report:", expensive_report(1), "| again:", expensive_report(1))
    print("sleeps recorded:", slept())
    print(f"total planned wait: {sum(slept()):.2f}s")


if __name__ == "__main__":
    main()
```

8. Написала тести замикань: фіксована політика повертає сталу паузу, експоненційна — геометричну прогресію з обмеженням, некоректні параметри

відхиляються, а `fake_sleep` записує паузи без очікування і не ділить стан між екземплярами:

```python
def test_fixed_delay_policy() -> None:
    policy = fixed_delay(0.25)
    assert [policy(i) for i in range(1, 5)] == [0.25, 0.25, 0.25, 0.25]
    assert fixed_delay(0.0)(7) == 0.0
    with pytest.raises(ValueError):
        fixed_delay(-1.0)


def test_exponential_delay_policy_and_cap() -> None:
    policy = exponential_delay(0.1, factor=2.0)
    assert [round(policy(i), 3) for i in range(1, 5)] == [0.1, 0.2, 0.4, 0.8]
    capped = exponential_delay(1.0, factor=3.0, max_delay=5.0)
    assert [capped(i) for i in range(1, 5)] == [1.0, 3.0, 5.0, 5.0]
    with pytest.raises(ValueError):
        exponential_delay(1.0, factor=0.0)


def test_fake_sleep_records_without_waiting() -> None:
    logs: list[str] = []
    sleep, slept = make_fake_sleep(logs.append)
    assert slept() == ()
    sleep(0.5)
    sleep(1.0)
    assert slept() == (0.5, 1.0)
    assert logs == ["(sleep 0.500s skipped)", "(sleep 1.000s skipped)"]
    other_sleep, other_slept = make_fake_sleep()
    other_sleep(9.0)
    assert slept() == (0.5, 1.0) and other_slept() == (9.0,)  # незалежний стан

```

9. Тести `timed`: фейковий годинник і список-логер замість реального часу, лог при винятку, збереження `__name__`, `__doc__` та `__wrapped__`:

```python
def test_timed_uses_clock_and_logger() -> None:
    ticks = iter([0.0, 0.012345])
    logs: list[str] = []

    @timed("work", unit="ms", logger=logs.append, clock=lambda: next(ticks))
    def work(x: int) -> int:
        """Квадрат."""
        return x * x

    assert work(5) == 25
    assert logs == ["[timed] work took 12.345 ms"]


def test_timed_logs_even_on_exception() -> None:
    ticks = iter([1.0, 3.0])
    logs: list[str] = []

    @timed(unit="s", logger=logs.append, clock=lambda: next(ticks))
    def boom() -> None:
        raise RuntimeError("x")

    with pytest.raises(RuntimeError):
        boom()
    assert logs == ["[timed] boom took 2.000 s"]


def test_timed_preserves_metadata() -> None:
    def original(a: int) -> int:
        """Doc."""
        return a

    wrapped = timed(logger=lambda _: None, clock=lambda: 0.0)(original)
    assert wrapped.__name__ == "original"
    assert wrapped.__doc__ == "Doc."
    assert wrapped.__wrapped__ is original  # type: ignore[attr-defined]

```

10. Тести `retries` для варіанта 14: фіксований backoff дає успіх на третій спробі з паузами `(0.5, 0.5)`, експоненційний з обмеженням — `[0.1, 0.2, 0.4, 0.5]`,

після останньої спроби виняток піднімається повторно, а зайва пауза не планується:

```python
def test_retries_fixed_backoff_succeeds_on_third_attempt() -> None:
    calls = {"n": 0}
    sleep, slept = make_fake_sleep()

    @retries(attempts=3, delay=fixed_delay(0.5), sleep=sleep)
    def flaky() -> str:
        calls["n"] += 1
        if calls["n"] < 3:
            raise RuntimeError("boom")
        return "ok"

    assert flaky() == "ok"
    assert calls["n"] == 3
    assert slept() == (0.5, 0.5)  # фіксований backoff, без реального сну


def test_retries_exponential_backoff_with_cap() -> None:
    calls = {"n": 0}
    sleep, slept = make_fake_sleep()

    @retries(attempts=5, delay=exponential_delay(0.1, 2.0, max_delay=0.5), sleep=sleep)
    def flaky() -> int:
        calls["n"] += 1
        if calls["n"] < 5:
            raise RuntimeError("boom")
        return calls["n"]

    assert flaky() == 5
    assert [round(s, 3) for s in slept()] == [0.1, 0.2, 0.4, 0.5]


def test_retries_reraises_after_last_attempt() -> None:
    logs: list[str] = []
    sleep, slept = make_fake_sleep()

    @retries(attempts=2, delay=fixed_delay(1.0), sleep=sleep, logger=logs.append)
    def always_fails() -> None:
        raise ValueError("nope")

    with pytest.raises(ValueError):
        always_fails()
    assert slept() == (1.0,)  # після останньої спроби пауза не планується
    assert any("fail after 2" in m for m in logs)

```

11. Додала тести, що політика отримує номери спроб `1, 2, 3`, обгортка зберігає

метадані й працює з однією спробою, а `attempts < 1` відхиляється:

```python
def test_retries_policy_receives_attempt_number() -> None:
    seen: list[int] = []
    calls = {"n": 0}

    def policy(attempt: int) -> float:
        seen.append(attempt)
        return 0.0

    @retries(attempts=4, delay=policy, sleep=lambda _: None)
    def flaky() -> None:
        calls["n"] += 1
        if calls["n"] < 4:
            raise RuntimeError

    flaky()
    assert seen == [1, 2, 3]


def test_retries_preserves_metadata_and_single_attempt() -> None:
    def task(x: int) -> int:
        """Doc."""
        return x

    wrapped = retries(attempts=1)(task)
    assert wrapped(3) == 3
    assert wrapped.__name__ == "task" and wrapped.__doc__ == "Doc."
    assert wrapped.__wrapped__ is task  # type: ignore[attr-defined]


def test_retries_invalid_attempts() -> None:
    with pytest.raises(ValueError):
        retries(attempts=0)
```

12. У `tests/test_scheduler.py` перевірила композиції декораторів: `timed` зовні `retries` робить один вимір на весь план повторів, а `cached` усередині `retries` не кешує невдалу спробу і не перераховує успішний результат:

```python
def test_timed_outside_retries_measures_whole_schedule() -> None:
    logs: list[str] = []
    tick = make_counter()
    sleep, slept = make_fake_sleep()
    calls = make_counter()

    @timed("job", unit="s", logger=logs.append, clock=lambda: float(tick()))
    @retries(attempts=3, delay=fixed_delay(2.0), sleep=sleep, logger=logs.append)
    def job() -> int:
        if calls() < 3:
            raise ConnectionError("down")
        return 42

    assert job() == 42
    assert slept() == (2.0, 2.0)
    assert logs[-1] == "[timed] job took 1.000 s"  # один вимір на весь план
    assert sum("attempt" in m for m in logs) == 2
    assert job.__name__ == "job"


def test_cached_inside_retries_is_not_recomputed_after_success() -> None:
    hits = make_counter()
    fails = make_counter()
    sleep, slept = make_fake_sleep()

    @retries(attempts=3, delay=exponential_delay(0.1), sleep=sleep)
    @cached()
    def report(day: int) -> str:
        hits()
        if fails() == 1:
            raise RuntimeError("first time fails")
        return f"report-{day}"

    assert report(1) == "report-1"
    assert report(1) == "report-1"
    assert hits() == 3  # 1 падіння + 1 успіх, другий виклик — кеш-хіт
    assert slept() == (0.1,)

```

13. Порівняла історії пауз фіксованої та експоненційної політик, через `monkeypatch` довела, що реальний `time.sleep` не викликається, і додала property-тест на Hypothesis: кількість запланованих пауз завжди дорівнює кількості невдач до успіху:

```python
def test_fixed_vs_exponential_total_wait() -> None:
    def run(policy: DelayPolicy) -> tuple[float, ...]:
        sleep, slept = make_fake_sleep()
        calls = make_counter()

        @retries(attempts=5, delay=policy, sleep=sleep)
        def task() -> None:
            if calls() < 5:
                raise RuntimeError

        task()
        return slept()

    fixed = run(fixed_delay(0.4))
    expo = run(exponential_delay(0.1, 2.0))
    assert fixed == (0.4, 0.4, 0.4, 0.4)
    assert [round(s, 3) for s in expo] == [0.1, 0.2, 0.4, 0.8]
    assert len(set(fixed)) == 1 and list(expo) == sorted(expo)


def test_no_real_sleep_is_called(monkeypatch: pytest.MonkeyPatch) -> None:
    import time

    monkeypatch.setattr(time, "sleep", lambda _: pytest.fail("real sleep called"))
    sleep, _ = make_fake_sleep()
    calls = make_counter()

    @retries(attempts=2, delay=fixed_delay(5.0), sleep=sleep)
    def task() -> str:
        if calls() < 2:
            raise RuntimeError
        return "done"

    assert task() == "done"


@given(st.integers(min_value=1, max_value=8), st.floats(min_value=0, max_value=2))
def test_property_sleep_count_is_failures_before_success(n: int, d: float) -> None:
    sleep, slept = make_fake_sleep()
    calls = make_counter()

    @retries(attempts=n, delay=fixed_delay(d), sleep=sleep)
    def task() -> int:
        if calls() < n:
            raise RuntimeError
        return n

    assert task() == n
    assert slept() == (d,) * (n - 1)
```

14. Запустила демонстрацію: обидві задачі відновлюються після невдач без реальних затримок, звіт будується один раз завдяки кешу, а планувальник

показує історію пауз і сумарний запланований час очікування:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab06> python app.py
[app] [retries] sync_task: attempt 1 failed: ConnectionError('sync endpoint busy')
[app] (sleep 0.500s skipped)
[app] [retries] sync_task: attempt 2 failed: ConnectionError('sync endpoint busy')
[app] (sleep 0.500s skipped)
[app] [timed] sync took 1.000 ms
sync: synced
[app] [retries] download_task: attempt 1 failed: TimeoutError('timeout https://example.org/data')
[app] (sleep 0.100s skipped)
[app] [retries] download_task: attempt 2 failed: TimeoutError('timeout https://example.org/data')
[app] (sleep 0.200s skipped)
[app] [retries] download_task: attempt 3 failed: TimeoutError('timeout https://example.org/data')
[app] (sleep 0.400s skipped)
[app] [timed] download took 1.000 ms
download: content of https://example.org/data
[app] building report for day 1
report: report-1 | again: report-1
sleeps recorded: (0.5, 0.5, 0.1, 0.2, 0.4)
total planned wait: 1.70s
```

15. Перевірила проєкт автоматичними інструментами: усі 28 тестів pytest проходять, `mypy --strict` не знаходить помилок типів, `ruff` та `black` не мають

зауважень до стилю:

```
C:\Users\maksi\Desktop\labs_functional_for_nastia\lab06> python -m pytest -v
============================= test session starts =============================
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\maksi\AppData\Local\Python\pythoncore-3.14-64\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: C:\Users\maksi\Desktop\labs_functional_for_nastia
configfile: pyproject.toml
plugins: hypothesis-6.170.0, platformdirs-4.12.3
collecting ... collected 28 items

tests\test_cached.py::test_cached_ttl_and_invalidate PASSED              [  3%]
tests\test_cached.py::test_cache_clear_and_size PASSED                   [  7%]
tests\test_cached.py::test_cached_preserves_metadata PASSED              [ 10%]
tests\test_cached.py::test_property_each_distinct_arg_computed_once PASSED [ 14%]
tests\test_closures.py::test_counter_keeps_state PASSED                  [ 17%]
tests\test_closures.py::test_counters_are_independent PASSED             [ 21%]
tests\test_closures.py::test_logger_uses_injected_sink PASSED            [ 25%]
tests\test_closures.py::test_fixed_delay_policy PASSED                   [ 28%]
tests\test_closures.py::test_exponential_delay_policy_and_cap PASSED     [ 32%]
tests\test_closures.py::test_fake_sleep_records_without_waiting PASSED   [ 35%]
tests\test_closures.py::test_property_counter_counts_calls PASSED        [ 39%]
tests\test_closures.py::test_property_exponential_is_monotone_and_bounded PASSED [ 42%]
tests\test_retries.py::test_retries_fixed_backoff_succeeds_on_third_attempt PASSED [ 46%]
tests\test_retries.py::test_retries_exponential_backoff_with_cap PASSED  [ 50%]
tests\test_retries.py::test_retries_reraises_after_last_attempt PASSED   [ 53%]
tests\test_retries.py::test_retries_only_for_listed_exceptions PASSED    [ 57%]
tests\test_retries.py::test_retries_policy_receives_attempt_number PASSED [ 60%]
tests\test_retries.py::test_retries_preserves_metadata_and_single_attempt PASSED [ 64%]
tests\test_retries.py::test_retries_invalid_attempts PASSED              [ 67%]
tests\test_scheduler.py::test_timed_outside_retries_measures_whole_schedule PASSED [ 71%]
tests\test_scheduler.py::test_cached_inside_retries_is_not_recomputed_after_success PASSED [ 75%]
tests\test_scheduler.py::test_fixed_vs_exponential_total_wait PASSED     [ 78%]
tests\test_scheduler.py::test_no_real_sleep_is_called PASSED             [ 82%]
tests\test_scheduler.py::test_property_sleep_count_is_failures_before_success PASSED [ 85%]
tests\test_timed.py::test_timed_uses_clock_and_logger PASSED             [ 89%]
tests\test_timed.py::test_timed_logs_even_on_exception PASSED            [ 92%]
tests\test_timed.py::test_timed_preserves_metadata PASSED                [ 96%]
tests\test_timed.py::test_timed_units PASSED                             [100%]

============================= 28 passed in 0.46s ==============================

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab06> python -m mypy .
Success: no issues found in 9 source files

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab06> python -m ruff check .
All checks passed!

C:\Users\maksi\Desktop\labs_functional_for_nastia\lab06> python -m black --check .
All done! ✨ 🍰 ✨
9 files would be left unchanged.
```

Висновок: Під час виконання лабораторної роботи я реалізувала замикання зі станом (`nonlocal`) і замикання-політики затримок, параметризовані декоратори `timed`, `retries` і `cached` з TTL та інвалідацією, а за варіантом 14 — планувальник повторів задач, у якому фіксований і експоненційний backoff передаються у `@retries` функцією `attempt → seconds`, а `sleep` замінено на `fake_sleep`, що лише записує паузи. Усі зовнішні ефекти (час, сон, логування) передаються як `Callable`, тому 28 тестів виконуються детерміновано без реальних затримок і підтверджують збереження метаданих через `functools.wraps`, роботу винятків і граничних параметрів та вплив порядку декораторів; mypy, ruff і black пройшли без зауважень.